PRN: 2124UCSM1057
Class: 3rd Year
Department: Cyber Security

Unit 3 – Cloud Security Assignment

Topic: Set Up a Web Application on Amazon EC2

1. Aim

To launch an Amazon EC2 instance, secure it with a properly scoped Security Group (SSH only from my own IP, HTTP for public web access), connect to it over SSH, install the Apache web server, deploy a simple HTML web application, and verify it in a browser, while documenting every command and security configuration.

2. Architecture Overview

Component	Choice
Cloud provider	Amazon Web Services (AWS)
Service	Amazon EC2 (Elastic Compute Cloud)
Operating system	Ubuntu Server 24.04 LTS
Instance type	t2.micro / t3.micro (Free Tier eligible)
Web server	Apache HTTP Server 2.4 (apache2)
Access method	SSH with key pair (.pem), no password login
Inbound ports	22 (SSH, my IP only), 80 (HTTP, public)

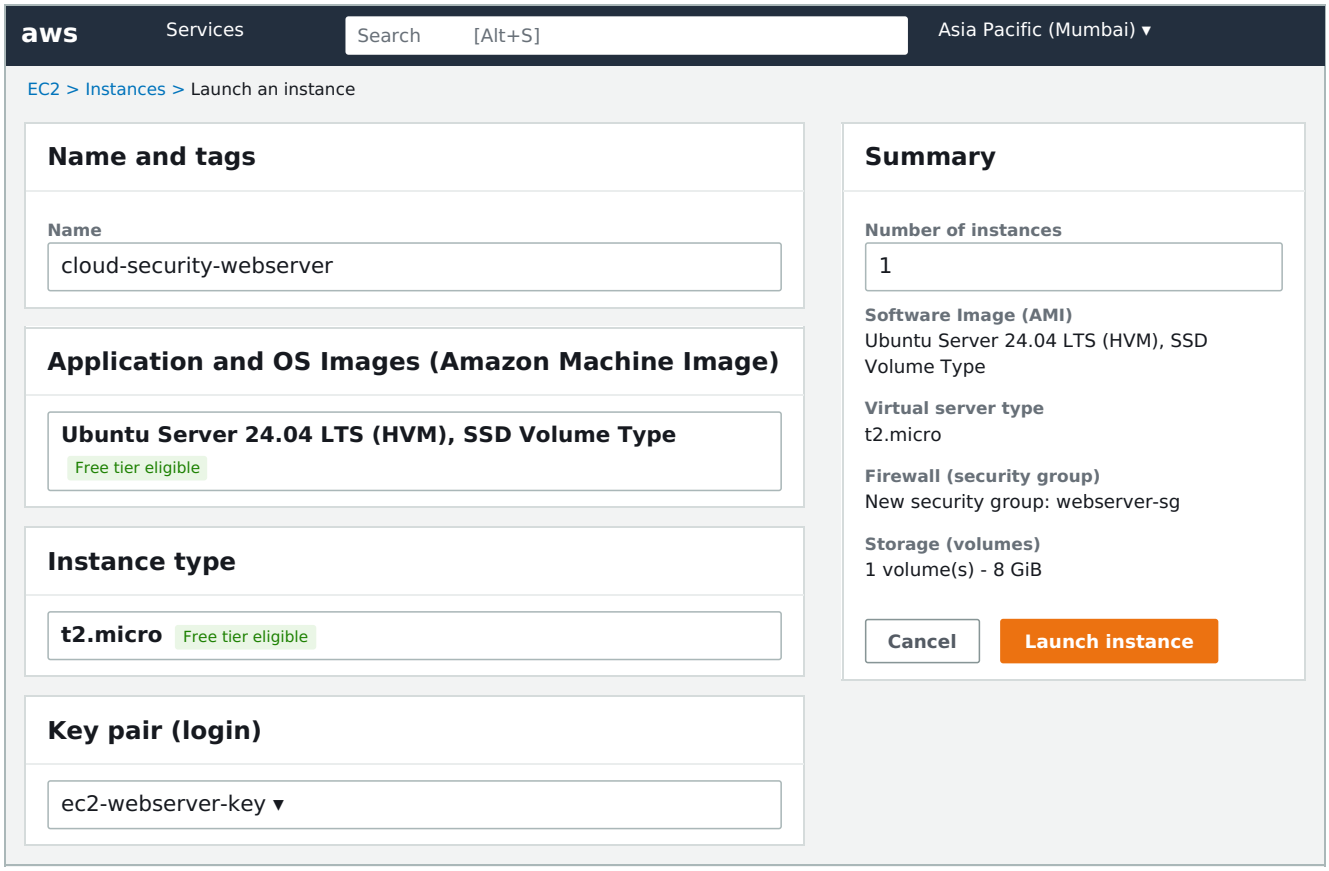
Traffic flow: Browser → Internet → Security Group (allows port 80) → EC2 instance → Apache serves `index.html`. Admin → SSH (port 22, restricted to my public IP) → EC2 instance.

3. Step 1 – Launch an EC2 Instance

1. Sign in to the **AWS Management Console** and open **EC2** (search “EC2” in the top bar).
2. Make sure the correct **Region** is selected (top right), for example *Asia Pacific (Mumbai) ap-south-1*.
3. Click **Launch instance** and configure:

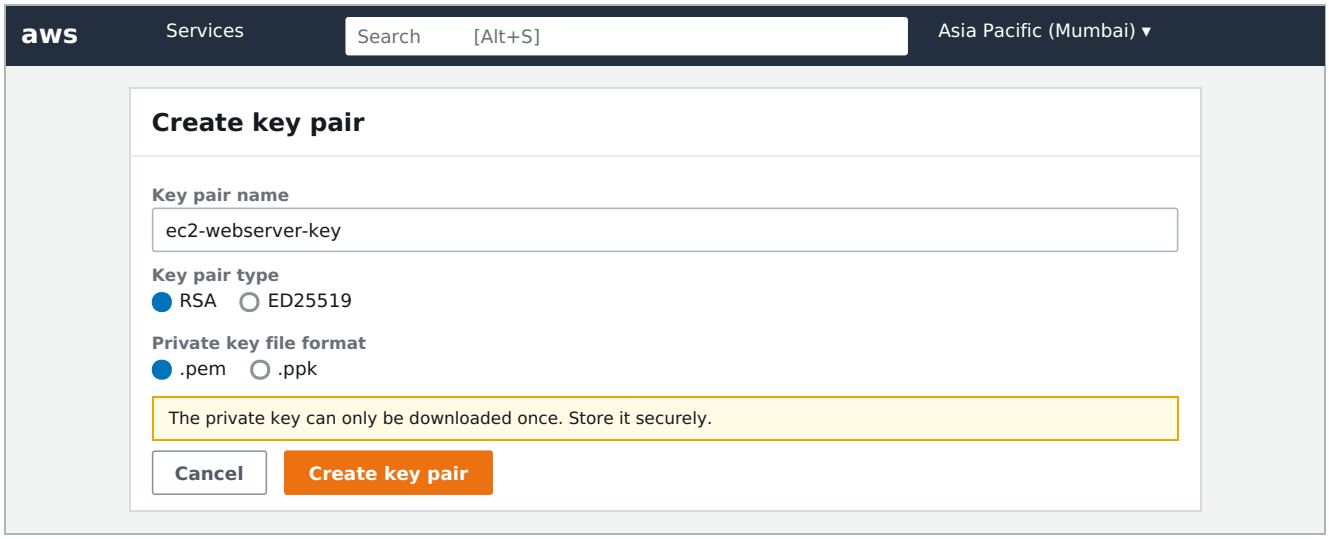
Setting	Value
Name	`cloud-security-webserver`
AMI	Ubuntu Server 24.04 LTS (64-bit x86)
Instance type	`t2.micro` (or `t3.micro`)
Key pair	Create new → name `ec2-webserver-key`, type **RSA**, format **.pem**
Storage	8 GiB gp3 (default)

4. **Download and safely store the `.pem` file.** It can only be downloaded once and is the only way to log in.



Illustrative mock-up of the console configuration, not a captured screenshot

Figure 1: Selecting the Ubuntu AMI and t2.micro instance type



Illustrative mock-up of the console configuration, not a captured screenshot

Figure 2: Creating the key pair used for SSH authentication

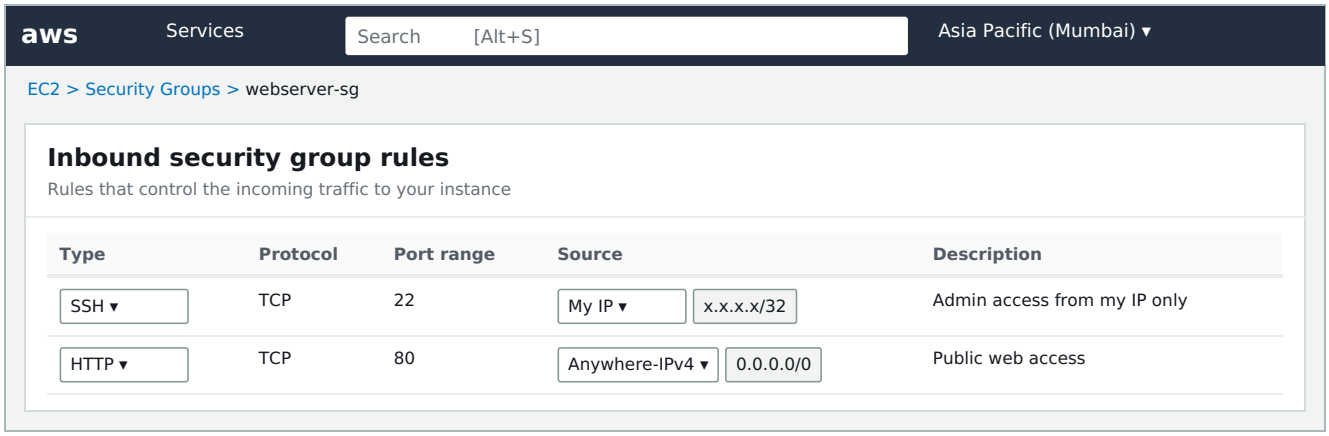
4. Step 2 – Configure the Security Group

Under **Network settings → Edit**, create a new security group named `webserver-sg` with the description *"Allow SSH from my IP and HTTP from anywhere"*.

Type	Protocol	Port	Source	Purpose
SSH	TCP	22	**My IP** (`x.x.x.x/32`)	Administrative access, only from my machine
HTTP	TCP	80	`0.0.0.0/0`	Public web access to the application

Security reasoning:

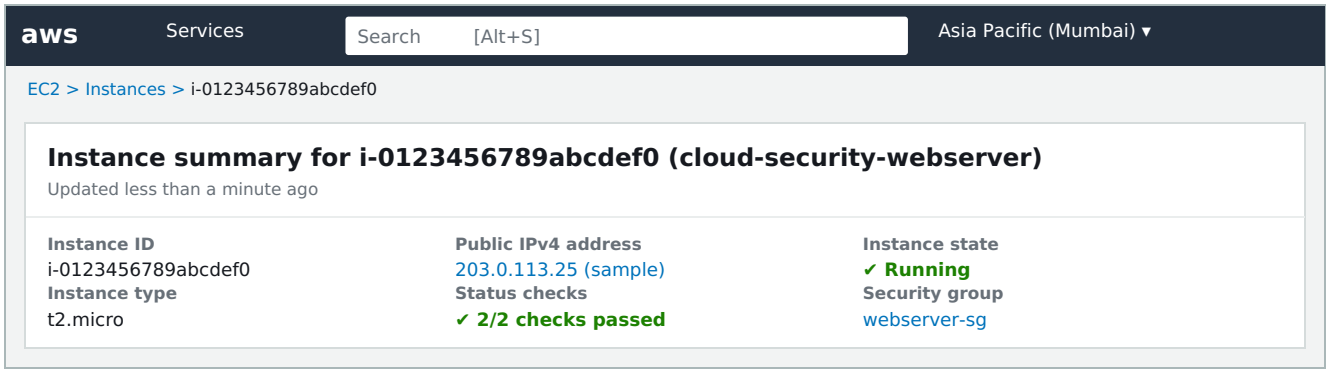
- SSH is **never** opened to `0.0.0.0/0`. Choosing **My IP** in the console fills in my current public IP as a `/32`, so only one address can even attempt to connect, which greatly reduces brute-force and scanning attacks.
- Only port 80 is exposed to the public, following the principle of least privilege. All other inbound ports are blocked by default, because Security Groups are *deny by default*.
- Outbound traffic is left at the default (all allowed) so the server can download updates and packages.



Illustrative mock-up of the console configuration, not a captured screenshot

Figure 3: Security Group inbound rules at launch time

Click **Launch instance**, then open **Instances** and wait until **Instance state = Running** and **Status check = 2/2 checks passed**. Note the **Public IPv4 address**.



Illustrative mock-up of the console configuration, not a captured screenshot

Figure 4: Running instance with its public IP address

5. Step 3 – Connect to the Instance Using SSH

On Linux / macOS / Windows PowerShell (OpenSSH):

```
# 1. Restrict key file permissions (SSH refuses keys that are too open)
chmod 400 ec2-webserver-key.pem

# 2. Connect (replace with your instance's public IPv4 address)
ssh -i ec2-webserver-key.pem ubuntu@<PUBLIC_IP>
```

Windows PowerShell – equivalent of chmod 400:

```
icacls .\ec2-webserver-key.pem /inheritance:r
icacls .\ec2-webserver-key.pem /grant:r "$($env:USERNAME):(R)"
ssh -i .\ec2-webserver-key.pem ubuntu@<PUBLIC_IP>
```

Type `yes` when asked to trust the host fingerprint. A successful login shows the Ubuntu welcome banner and the prompt `ubuntu@ip-172-31-x-x:~$`.

> **Tip:** If the connection times out, the usual causes are (a) your public IP changed so the SSH rule no

longer matches, or (b) the instance is not running yet.

[Insert Screenshot 5: Terminal showing successful SSH login (Ubuntu welcome banner)]

Figure 5: Successful SSH connection to the EC2 instance

6. Step 4 – Install and Configure the Apache Web Server

```
# Update the package index and upgrade installed packages
sudo apt update && sudo apt upgrade -y

# Install Apache
sudo apt install apache2 -y

# Start Apache now and enable it to start on every boot
sudo systemctl start apache2
sudo systemctl enable apache2

# Verify the service is running
sudo systemctl status apache2
```

The status output should show **active (running)** in green. Press `q` to exit.

```
# Check Apache version
apache2 -v

# Confirm Apache is listening on port 80
sudo ss -tlnp | grep :80
```

[Insert Screenshot 6: “systemctl status apache2” showing active (running)]

Figure 6: Apache service running on the instance

Alternative (Nginx): `sudo apt install nginx -y && sudo systemctl enable --now nginx`. The web root for Nginx is also `/var/www/html`.

7. Step 5 – Deploy a Simple HTML Web Application

Apache serves files from `/var/www/html`. Replace the default page with a custom application:

```
sudo nano /var/www/html/index.html
```

Paste the following content, then save with `Ctrl+O`, `Enter`, and exit with `Ctrl+X`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cloud Security - EC2 Web App</title>
  <style>
    body { font-family: Arial, sans-serif; background:#0f172a; color:#e2e8f0;
           text-align:center; padding:60px; }
    .card { background:#1e293b; display:inline-block; padding:30px 50px;
            border-radius:12px; box-shadow:0 4px 20px rgba(0,0,0,.4); }
    h1 { color:#ff9900; }
  </style>
</head>
<body>
  <div class="card">
    <h1>Hello from Amazon EC2!</h1>
    <p>Web application deployed on Ubuntu with Apache.</p>
    <p><strong>Unit 3 - Cloud Security Assignment</strong></p>
    <p>PRN: 2124UCSM1057 | Cyber Security</p>
  </div>
</body>
</html>
```

Set correct ownership and permissions, and validate the configuration:

```
sudo chown -R www-data:www-data /var/www/html
sudo chmod -R 755 /var/www/html

# Test Apache configuration syntax, then reload
sudo apache2ctl configtest
sudo systemctl reload apache2
```

`Syntax OK` confirms the configuration is valid.

[Insert Screenshot 7: nano editor with index.html, or “cat /var/www/html/index.html” output]

Figure 7: HTML application file created in the web root

8. Step 6 – Verify the Application in a Browser

From the server itself:

```
curl -I http://localhost
```

Expected: `HTTP/1.1 200 OK`.

From my own computer’s browser: open `http://<PUBLIC_IP>` (use **http**, not https, because no TLS certificate is configured).

The custom “Hello from Amazon EC2!” page should load, which proves that the instance is running, Apache is serving content, and the Security Group correctly allows HTTP on port 80.

[Insert Screenshot 8: Browser showing the web page at http://PUBLIC_IP]

Figure 8: Web application served from the EC2 instance

9. Security Configurations Summary

#	Control	Implementation	Benefit
1	Network firewall	Security Group `webserver-sg` allows only ports 22 and 80	Minimizes attack surface
2	Restricted SSH	Port 22 limited to my IP (`/32`)	Blocks internet-wide brute force
3	Key-based authentication	RSA key pair; password login disabled by default on the AMI	Not guessable like passwords
4	Key file protection	`chmod 400` on the `.pem` file	Prevents key leakage to other users
5	Least-privilege web files	Owned by `www-data`, mode 755	Web server cannot modify system files
6	Patching	`apt update && apt upgrade`	Fixes known vulnerabilities
7	Non-root login	Log in as `ubuntu`, use `sudo` only when required	Limits damage and gives audit trail

Additional hardening (recommended)

```
# Hide Apache version and OS details in HTTP headers / error pages
sudo nano /etc/apache2/conf-available/security.conf
#   ServerTokens Prod
#   ServerSignature Off
sudo systemctl reload apache2

# Enable automatic security updates
sudo apt install unattended-upgrades -y
sudo dpkg-reconfigure --priority=low unattended-upgrades
```

Further improvements for a production environment: enforce **IMDSv2** on the instance (Actions → Instance settings → Modify instance metadata options → *IMDSv2 required*), use an **Elastic IP** so the address does not change on restart, add **HTTPS** with a free Let's Encrypt certificate (`certbot`) once a domain name is attached, and attach an **IAM role** instead of storing AWS credentials on the server.

[Insert Screenshot 9: Security Group "Inbound rules" tab after launch]

Figure 9: Final inbound rules attached to the instance

10. Troubleshooting

Problem	Likely cause	Fix
SSH: connection timed out	My public IP changed, or SG rule missing	Edit SG inbound rule, set SSH source to **My IP** again
SSH: permission denied (publickey)	Wrong username or key	Use `ubuntu@` for Ubuntu AMIs and the correct `.pem`
SSH: unprotected private key file	Key permissions too open	`chmod 400 key.pem`
Browser: page does not load	Port 80 not open, or using https	Add HTTP rule; use `http://`
Browser: shows Apache default page	index.html not replaced	Re-create `/var/www/html/index.html`, hard refresh
Apache not running	Service stopped/misconfigured	`sudo systemctl restart apache2`, `sudo apache2ctl configtest`

11. Cleanup (to Avoid Charges)

After the assignment is evaluated: **EC2 → Instances → select instance → Instance state → Terminate instance**. Also release any Elastic IP and delete unused volumes and security groups.

12. Conclusion

An Amazon EC2 instance was launched and secured with a least-privilege Security Group that allows SSH only from a single trusted IP and HTTP for public web access. After connecting over key-based SSH, the Apache web server was installed and configured, and a custom HTML application was deployed and verified in a browser. The exercise demonstrates the core cloud security practices of network segmentation, restricted administrative access, key-based authentication, least-privilege file permissions, and regular patching.